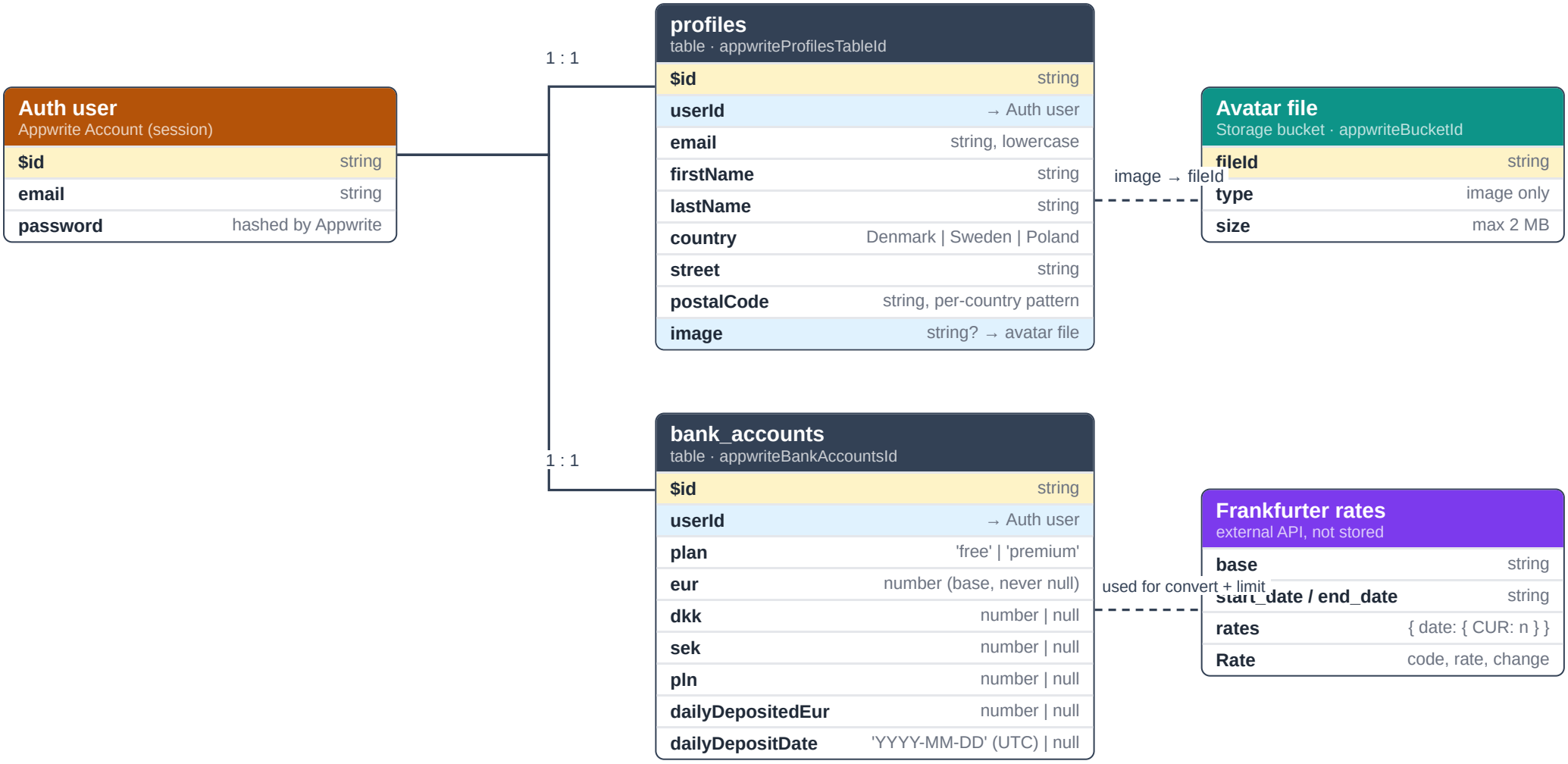
Bank App: data model

Appwrite Auth + TablesDB + Storage, with live FX rates from the Frankfurter API. Derived from `src/app/core/models`, the services and `environment.models.ts`.

key / identifier | reference to another entity



Fields, rules and limits

What each piece of the model is for, and the business rules the code enforces.

Entity	Purpose	Notable rules (from the code)
profiles	Personal data for each registered user. Looked up by `userId` after login and by `email` when sending money.	Email trimmed + lowercased on create. Rows get owner-only permissions. Recipient lookup selects only `userId, firstName, lastName, image`.
bank_accounts	One row per user holding all balances as columns, one per supported currency.	`null` = sub-account not opened. EUR cannot be removed. New accounts start as `plan: free`, `eur: 0`. Amounts rounded to 2 decimals.
Avatar file	Profile picture, referenced by `profiles.image`.	On change, the new file is uploaded first and the old one deleted afterwards.
Frankfurter rates	Live exchange rates, fetched over HTTP; never persisted.	Used to convert between own currencies and to express deposits in EUR for the limit.

Plans & daily deposit limit

Plan	Daily limit (EUR)
free	100,000
premium	500,000

- Each deposit is converted to EUR and added to `dailyDepositedEur`.
- If `dailyDepositDate` isn't today (UTC), usage counts as 0.

Supported currencies & countries

Code	Locale
eur	en-IE
dkk	da-DK
sek	sv-SE
pln	pl-PL

Country	Postal code format
Denmark	4 digits
Sweden	3 digits, optional space, 2 digits
Poland	NN-NNN

Money operations

- **Deposit:** adds to one currency, checks the daily limit.
- **Withdraw:** fails if balance is insufficient.
- **Convert:** moves between two of your own currencies at the live rate.
- **Send:** withdraws from sender, credits the recipient's same-currency column; recipient must exist and have that currency open. On failure the sender is refunded.